



THE UNIVERSITY OF  
**AUCKLAND**  
Te Whare Wānanga o Tāmaki Makaurau  
NEW ZEALAND

# COURSE CONCLUSION

---

COMPSCI 230 OBJECT-ORIENTED SOFTWARE DEVELOPMENT

PARAMVIR SINGH

# AGENDA

---

- SET Evaluations
- Another Piazza Poll !
- Exam Format and Tips
- Design pattern categorization
- Learning outcomes achieved
- Topics not covered
- A step in the 'Software Development' pathway
- Beyond this course
- What we learnt?
- Your feedback?

# PLEASE HELP US IMPROVE!

- You AND ONLY You know how we did in the course.
- Your input plays a huge role in enabling improvement.
- There would be things that you would like to change and things that you would NEVER like to change.
- So...
- Please fill in your SET evaluations



## TIPS for PROVIDING HELPFUL FEEDBACK in SET COURSE and TEACHING EVALUATIONS

- Your feedback provides really important information that helps teachers to improve the course and its delivery for future students.
- Don't forget to complete evaluations before the closing date! Teachers and course coordinators do want to hear your views on the course.

### Be constructive!

- Comments that are directly related to your learning experience will really increase the value of your feedback.
- Using **improper language** or including **personal comments about an individual** is highly inappropriate, and will undermine your feedback.
- **Abusive or threatening comments** are unacceptable. SET evaluations are **confidential**, and the identity of individuals will never be revealed in normal reporting processes. However, the University has a commitment to ensure all members of its community are treated with dignity and respect. Students who use SET to comment about individuals in a way contrary to this commitment may be subject to disciplinary proceedings.  
See: [Prevention of Bullying, Harassment and Discrimination Guidelines](#).

### Be specific!

- You can provide great examples of how you think your teacher or the way the course is organised has helped (or not helped!) your learning.
- Specific, constructive, feedback is helpful whether it is constructively critical, or positive.

# PIAZZA POLL (OPEN DURING THE LECTURE)

---

- Which of the **design patterns** covered in this course did you enjoy the most?
  - Composite
  - Template
  - Observer
  - Adapter
  - MVC
  - Singleton
  - Immutable
  - Factory

# EXAM TOPICS AND FORMAT

---

- **Exam Format**
  - Time duration: 3 hours
  - Total Marks: 50
  - Question Format: MCQs, Embedded and Programming
  - Mode: In-person CR-based
  - Similar format to past exams
  - Part – I 30%, Part-2 70%
  - Examination Preparation Guidelines

# THE 'GANG OF FOUR'

## Gang of Four (GoF)

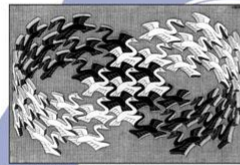


■ Ralph Johnson, Richard Helm, Erich Gamma, and John Vlissides (left to right)

## Design Patterns

Elements of Reusable  
Object-Oriented Software

Erich Gamma  
Richard Helm  
Ralph Johnson  
John Vlissides



Cover art © 1994 M.E. Eicher / Corbis Art - Baum - Holland. All rights reserved.

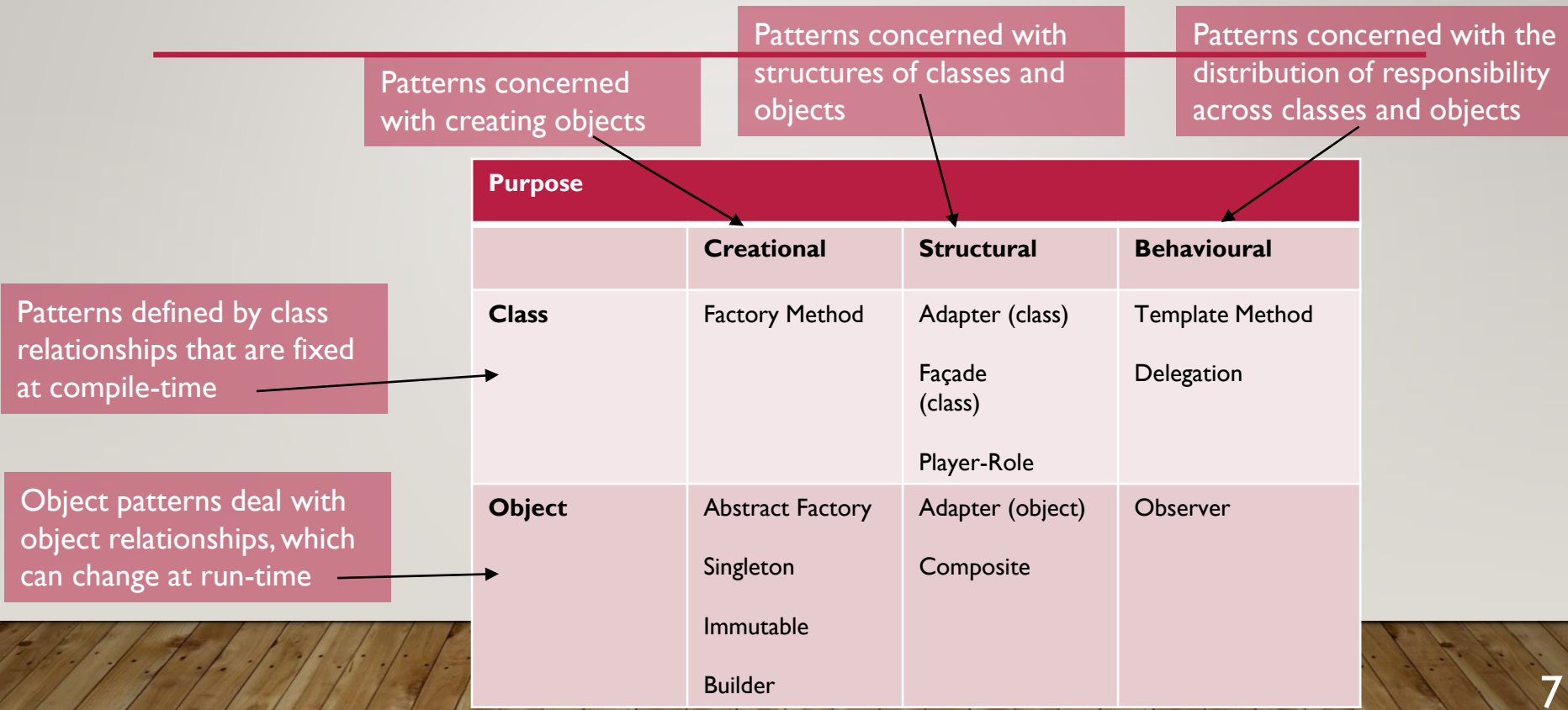
Foreword by Grady Booch

ADDISON-WESLEY PROFESSIONAL COMPUTING SERIES





# DESIGN PATTERN CLASSIFICATION



# LEARNING OUTCOMES ACHIEVED

---

- **OO Programming:** describe and use the features typically offered by an object-oriented programming language, including support for classes, visibility, inheritance, interfaces, polymorphism and dynamic binding
- **OO Design:** explain and apply key design principles of object-oriented software development, including separation of concerns, abstraction, information hiding, programming to interfaces, coupling and cohesion, resilience to change, and reuse
- **Software Modelling:** create simple OO design models including class models, object models and sequence models.
- **Design Patterns:** develop a small graphical user interface (GUI) based application in an object-oriented programming language; Assess and improve software design decisions based on object-oriented design objectives and principles
- **Frameworks:** describe important concepts of programming frameworks, including APIs, inversion of control, concurrency and event-driven programming



# TOPICS NOT COVERED IN PART-2

---

- UML Modelling
  - State Diagrams, Component Diagrams, Deployment diagrams, etc.
  - Object Constraint Language (OCL)
- Design Patterns
  - Builder, Abstract Factory, Proxy, Façade, Decorator, Strategy, Flyweight, etc.
- Design Principles
  - Common Closure, Release Reuse, etc.
- Refactoring
  - Refactoring activities (e.g. move method, extract class. Refer books by Robert Martin, Martin Fowler and others)
- Design Objectives
  - Portability, Reliability, Usability, etc.
- Advanced Swing Components and JavaFX
  - JTabbedPane, JDialog, Form components (A4), etc.
- Advanced Multithreading concepts
  - Producer-Consumer Problems, Locks, etc.

# A STEP IN THE SOFTWARE DEVELOPMENT MODULE

## Software Development

Gain the skills required to plan, design and write computer programs using standard computer programming languages. These skills are valuable for almost all disciplines, and are essential for those disciplines that involve processing or analysis of data.

### The courses you can take

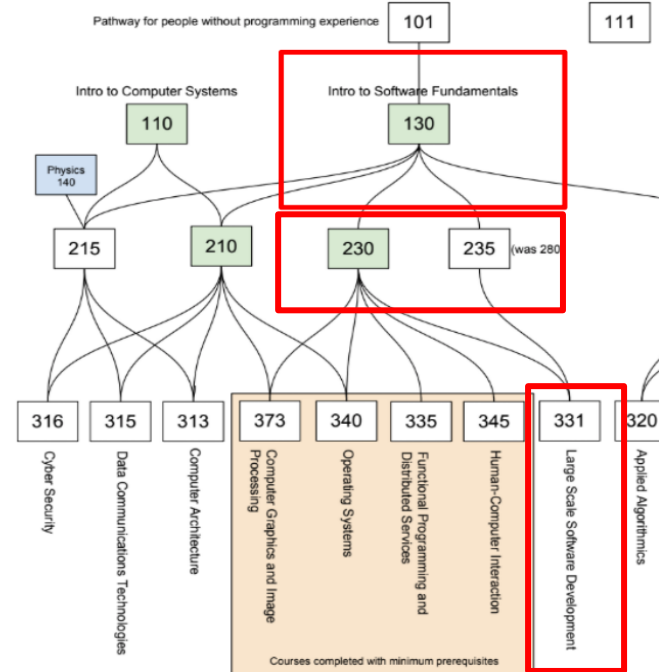
A module contributes 45 points to the overall structure of your undergraduate Science programme.

Each course is worth 15 points, and you must take three courses from those listed below.

#### 45 points from:

- [COMPSCI 101 Principles of Programming](#)
- [COMPSCI 130 Introduction to Software Fundamentals](#)
- [COMPSCI 230 Programming Techniques](#)
- [COMPSCI 235 Software Development Methodologies](#)
- [COMPSCI 331 Large-Scale Software Development](#)

## Computer Science Major



# BEYOND THE COURSE

---

- **Software Development**

- Extend your assignment/lab applications
- Keep learning new elements of software design and architecture
- Explore professional software development at companies like Google, Facebook, Microsoft, etc. and less known or start-up companies too.
- Challenges
  - Use of Patterns in Java API
  - Measuring the impact of design patterns on design
- Work hard on practicing –
  - independent (from instructors/tutors) and continuous learning
  - creative problem solving
  - solution-oriented and collegial approach to learning

- **Research Conferences**

- International Conference on Software Engineering (ICSE)
- International Conference on Software Maintenance and Evolution (ICSME)
- Foundations of Software Engineering Conference (FSE)
- Mining Software Repositories (MSR)
- And many more.... (Google Top SE conferences)

# WHAT YOUR TEACHERS LEARNT?

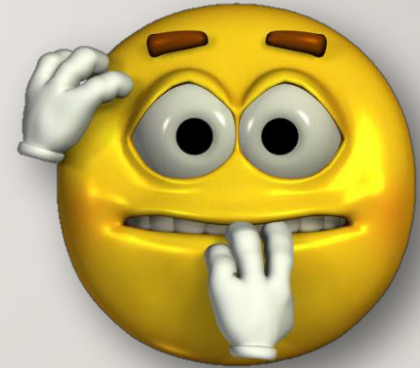
---

- CodeRunner Qs have had minor typos at times, which we continue to fix.
- CodeRunner is problematic for setting/answering GUI-based questions
- More practice exercises are always good
- How to get more students attending lectures and labs?
- *There will certainly be more added to this....*

# YOUR FEEDBACK (LIVE)

---

- Do you want to say something?



# Thank You !

---

*"You were great!"*

*We hope you enjoyed the course and we'll see you around :) "*

*- 230 teaching team*